

to synchronise the write access to the shared memory. All these IPC mechanisms can be used within a system, but to communicate between two different systems (with different IP addresses), socket programming is used.

Message queues can be used for communicating between processes or threads. Linux provides two choices for message queues — System V and POSIX. System V message queue is designed for server applications, whereas POSIX message queue is designed for real-time applications. The POSIX MQ has several advantages over the Sys V MQ implementation. Since it is a file based interface to the applications, we can set a priority for each message, which will determine the position of the message in the queue. Asynchronous notification of message arrivals can be done through signals or thread creation, and setting a timeout option will avoid indefinite waiting for sending or receiving messages. These features provide an efficient, predictable and deterministic mechanism to pass data between cooperating processes, which is very important for real-time applications.

In this article, we will discuss POSIX message queues in detail, including the creation of a message queue, sending and receiving a message, setting the priority of a message, and setting timeout to receive a message in the message queue.

System V message queue

Message queues in the System V API are associated with a message queue ID. These IDs are positive integers. The System V message queue is described by the *struct msqid_ds* structure specified in the header `<sys/msg.h>`. The members of the structure are shown in Figure 1.

ipc_perm msg_perm: This is a structure that describes the message queue access permissions.

msgqnum_t msg_qnum: This acknowledges to the process the number of messages that are currently in the message queue.

msglen_t msg_qbytes: This describes the ceiling size of messages that can be placed on the message queue. It is in bytes.

pid_t msg_lspid: This is the latest process's ID that used the *msgsnd()* system call to send messages.

pid_t msg_lrpid: This is the latest process's ID that used the *msgrcv()* system call in order to receive messages.

```
struct msqid_ds {
    struct ipc_perm msg_perm;
    msgqnum_t      msg_qnum; /* no of messages on queue */
    msglen_t       msg_qbytes; /* bytes max on a queue */
    pid_t          msg_lspid; /* PID of last msgsnd(2) call */
    pid_t          msg_lrpid; /* PID of last msgrcv(2) call */
    time_t         msg_stime; /* last msgsnd(2) time */
    time_t         msg_rtime; /* last msgrcv(2) time */
    time_t         msg_ctime; /* last change time */
};
```

Figure 1: Attribute structure of System V message queues

time_t msg_stime: Timestamp when the most recent *msgsnd()* system call was made.

time_t msg_rtime: Timestamp when the most recent *msgrcv()* system call was made.

time_t msg_ctime: Timestamp when the most recent change was made to a member of the *msqid_ds* structure.

System V messages have an associated length and message type unlike in pipes, where data is pumped through the pipe in a byte stream. Messages can be associated with message types, and the sending and receiving processes can place and retrieve these messages, respectively, by a specified message type.

A structure can be defined to describe the specifics of a message a process is trying to send. This structure is used to define the message buffer accessed and manipulated by the sending and receiving processes.

```
Struct msgbuf {
    long mtype; /* message type, must be > 0 */
    char mtext[1]; /* message data */
};
```

After the creation of message queue, you can view the information by executing the command:

```
ipcs -q.
```

Figure 2 shows the message queue details and limitations.

By default, the ceiling of the amount of messages that a System V message queue can contain in Linux is 8192 messages with a maximum size of 16384 bytes. These values are optimised for better performance. But one can alter this information depending on the requirements by changing the kernel parameters, which are stored in */proc/sys/kernel/msgmax* or *msgmnb* files.

Introduction to POSIX message queue

The library function and the corresponding system call details of the POSIX message queue's API can be found in the *mq_overview* man page. The kind of message queue you use may depend largely on the purpose of the application. The POSIX standards are the latest, and allow for cleaner interfaces to exchange messages amongst processes.

```
raju@ostl:~/osfy/posix/final$ ipcs -q

----- Message Queues -----
key      msqid      owner      perms      used-bytes   messages

raju@ostl:~/osfy/posix/final$ ipcs -lq

----- Messages Limits -----
max queues system wide = 32000
max size of message (bytes) = 8192
default max size of queue (bytes) = 16384
```

Figure 2: System V message queue system wide limits